`optimize_anything`: A Universal API for Optimizing any Text Parameter

February 18, 2026 • Lakshya A Agrawal*, Donghyun Lee*, Wenjie Ma, Karim Elmaaroufi, Shangyin Tan, Sanjit A. Seshia, Koushik Sen, Dan Klein, Ion Stoica, Joseph E. Gonzalez, Omar Khattab, Alexandros G. Dimakis, Matei Zaharia
* Equal Contribution

Today we are introducing `optimize_anything`, a declarative API that optimizes any artifact representable as text (e.g., code, prompts, agent architectures, vector graphics, configurations). It extends GEPA (Genetic-Pareto, our state-of-the-art LLM prompt optimizer) far beyond prompts. You declare what to optimize and how to measure it; the system handles the search. Testing it across several domains, we find `optimize_anything` consistently matches or outperforms domain-specific tools, including some purpose-built for each task. With one API, you can:

- create agent skills achieving near-perfect Claude Code task completion 47% faster,
- optimize cloud scheduling policies that cut costs by 40%, beating expert heuristics,
- find detailed system prompts to boost GPT's math reasoning accuracy,

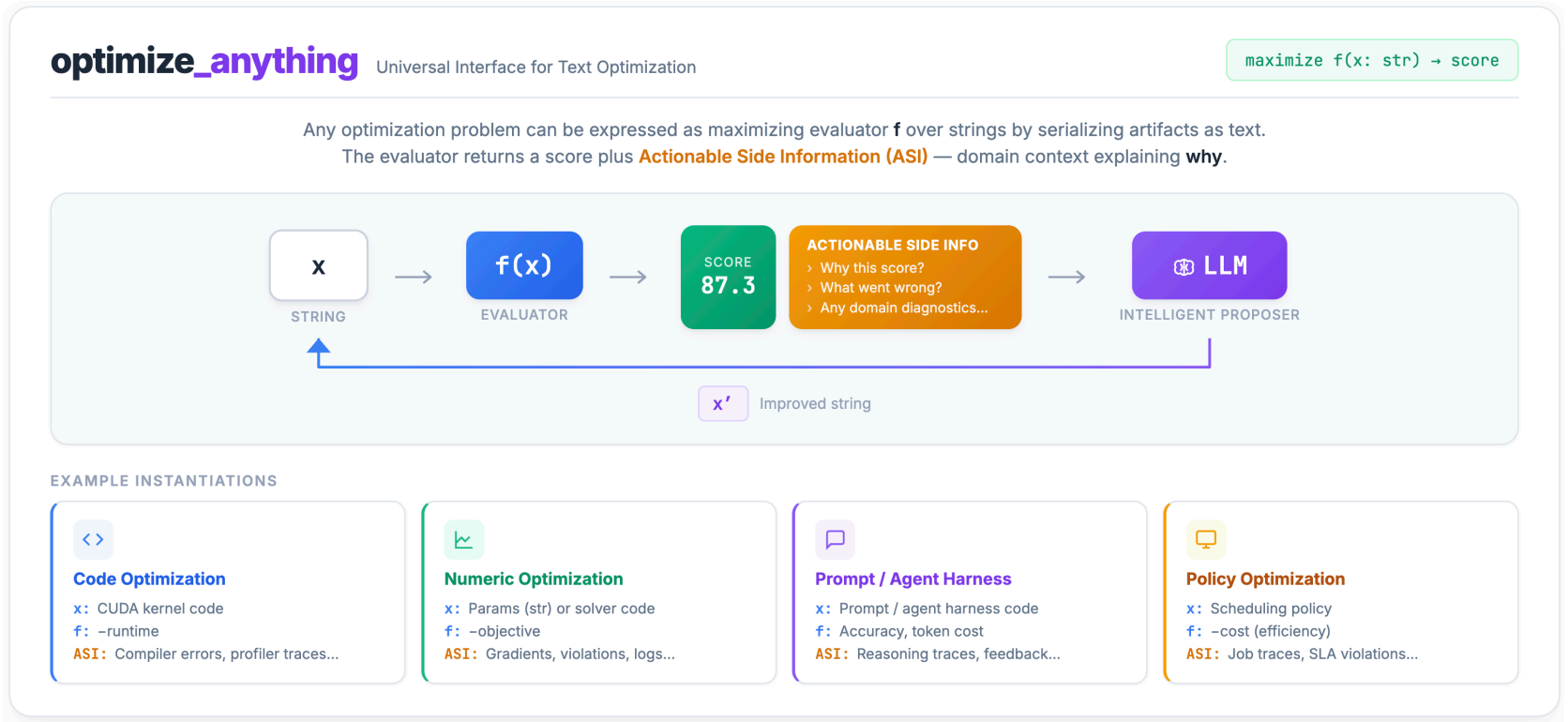
- discover bespoke agent harnesses that nearly triple Gemini Flash's ARC-AGI accuracy,
- write custom solvers to match and exceed Optuna in blackbox mathematical optimization,
- and... model a 3D unicorn.

Zero-shot attempt from Claude Opus 4.6

Optimized by `optimize_anything`

The key insight is that a surprisingly wide range of problems can be formulated as optimizing a text artifact: speeding up a CUDA kernel, tuning a scheduling policy, refining a prompt template, or redesigning an agent architecture. If it can be serialized to a string and its quality measured, an LLM can reason about it and propose improvements.

Where prior LLM-evolution frameworks like AlphaEvolve, OpenEvolve, and ShinkaEvolve expose concepts like island topologies[1], prompt samplers[2], and cascade evaluation stages[3], `optimize_anything` strips the interface down to its essence — and goes further by unifying **three optimization modes** (single-task search, multi-task search, and generalization) under one declarative API. While prior systems operate exclusively in single-task mode, `optimize_anything` enables optimization tasks they cannot directly express like discovering agent architectures from scratch, learning prompts that generalize to unseen examples, and optimizing coding agent skills that transfer across models.



Evaluate a text artifact, capture diagnostic feedback (ASI), and let an LLM propose targeted improvements.

Code, prompts, configs, agent architectures — if you can measure it, `optimize_anything` can optimize it.

The `optimize_anything` API

The Simplest Form

At its core, the API requires just two things: an artifact (or a description of what you want) and an evaluator.

```python
import gepa.optimize_anything as oa

def evaluate(candidate: str) -> float:
    score, diagnostic = run_my_system(candidate)
    oa.log(f"Error: {diagnostic}")  # captured as ASI
    return score

# Start from an existing artifact…
result = oa.optimize_anything(
    seed_candidate="<your initial artifact>",
    evaluator=evaluate,
)

# … or just describe what you need.
result = oa.optimize_anything(
    evaluator=evaluate,
    objective="Generate a Python function `reverse()` that reverses a string.",
)

print(result.best_candidate)
```

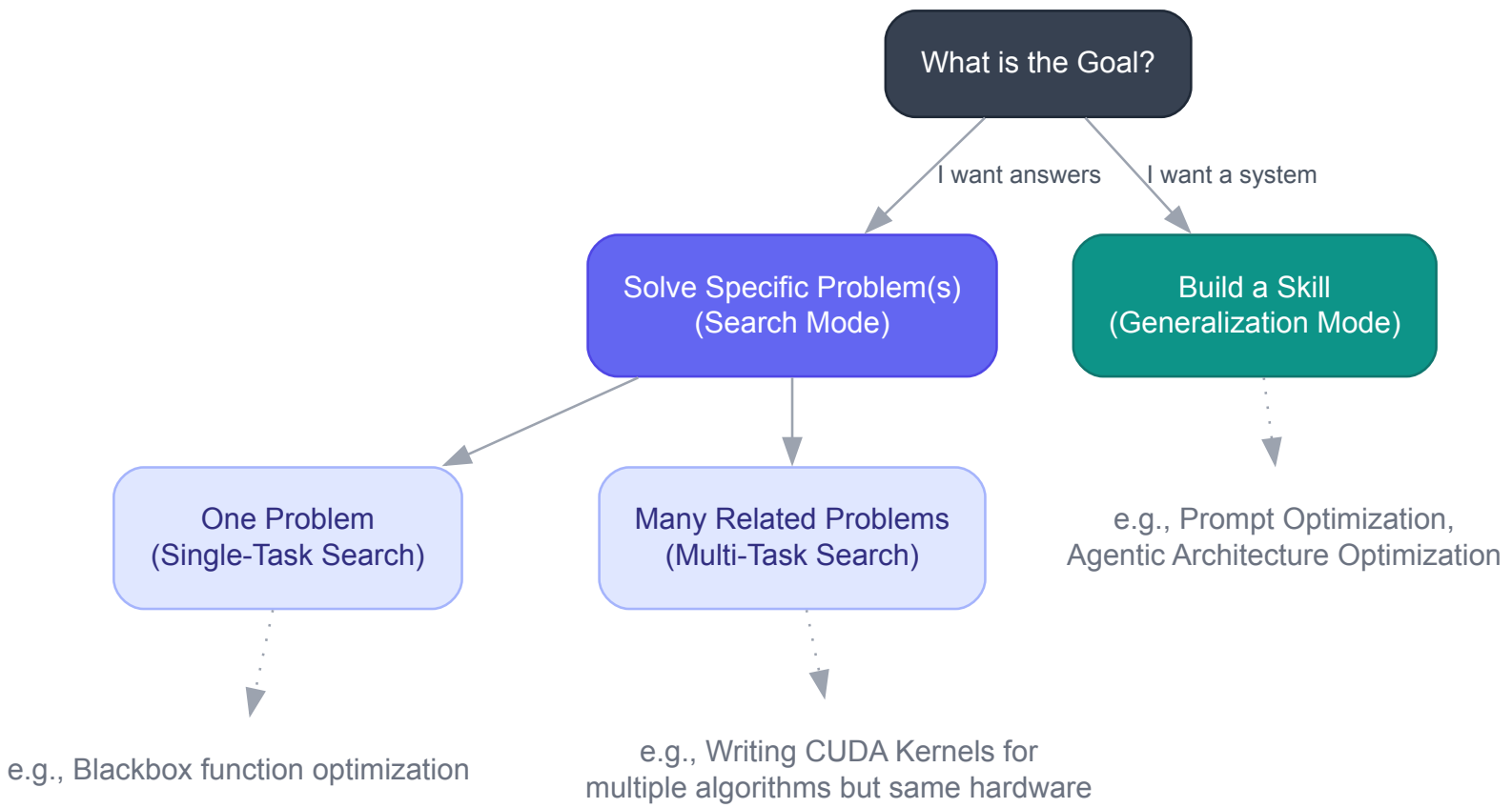
That's it. The evaluator takes a candidate string and returns a score (higher is better). `oa.log()` works just like `print()`, but routes output to the LLM proposer as **Actionable Side Information (ASI)** — diagnostic

feedback the proposer reads during reflection. For richer diagnostics, return a structured dictionary alongside the score:

```python
def evaluate(candidate: str) -> tuple[float, dict]:
    result = execute_code(candidate)
    return result.score, {
        "Error": result.stderr,
        "Output": result.stdout,
        "Runtime": f"{result.time_ms:.1f}ms",
    }
```

ASI can be open-ended text, structured data, multi-objectives (through `scores`), or even **images** (via `gepa.Image`) for vision-capable LLMs; anything that would help an expert understand the artifact and diagnose failures. We'll see ASI in action in the SVG demo, then unpack why it matters.

One Interface, Three Optimization Modes



The three optimization modes supported in `optimize_anything`: single-task search, multi-task search, and generalization.

`optimize_anything` unifies three distinct optimization paradigms under one API, determined by whether you provide a `dataset` and `valset`:

1. Single-Task Search: "Solve one hard problem." No dataset needed; the candidate *is* the solution, and the evaluator scores it directly (no `example` argument). For example, in circle packing, the artifact is the packing algorithm code and the evaluator returns the score plus geometric diagnostics as ASI. This is the mode that prior LLM-evolution frameworks like AlphaEvolve and OpenEvolve operate in.

```
oa.optimize_anything(seed_candidate=..., evaluator=...)
```

2. Multi-Task Search: "Solve a batch of related problems with cross-transfer." You provide a `dataset` of related tasks; insights from solving one help solve the others. For example, in CUDA kernel generation, each task is a PyTorch operation to accelerate on the same hardware, and the evaluator compiles and benchmarks the kernel returning compiler errors and profiler traces as ASI. Even though the kernels perform different computations, multi-task mode converges faster and solves more problems across all speedup thresholds than dedicated single-task optimization, thanks to cross-transfer of optimization patterns. No prior LLM-evolution framework supports this mode.

```
oa.optimize_anything(seed_candidate=..., evaluator=..., dataset=tasks)
```

3. Generalization: "Build a skill that transfers to unseen problems." You provide both a training `dataset` and a held-out `valset`; the optimized artifact (a prompt, an agent, a policy) must generalize to unseen examples. This is the mode that GEPA's prompt optimization operates in. `optimize_anything` generalizes the pattern to any text artifact, not just prompts, abstracting over traditional machine learning and program synthesis. For example, in agent architecture discovery, the artifact is the entire agent, the dataset and valset are ARC-AGI puzzles, and the evaluator runs the agent and returns its errors as ASI. The optimized agent improves from 32.5% to 89.5% on the test set (+57 percentage points). The same mode also powers cloud scheduling policy discovery, where the artifact is an algorithm that must generalize across unseen infrastructure scenarios.

```
oa.optimize_anything(seed_candidate=..., evaluator=..., dataset=train, valset=val)
```

The full API signature:

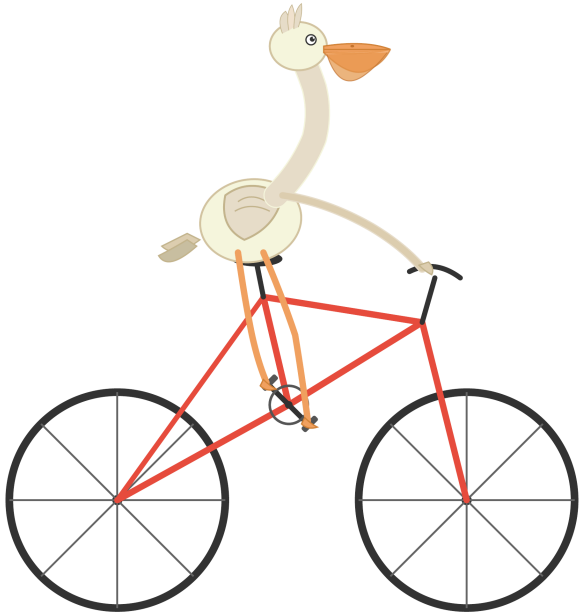
```
def optimize_anything(
    seed_candidate: str | dict[str, str] | None = None,  # Starting artifact (or None for seedless)
    evaluator: Callable,                    # Score + ASI
    dataset: list | None = None,            # Training examples (modes 2 & 3)
    valset: list | None = None,             # Validation set (mode 3)
    objective: str | None = None,           # What to optimize for (natural language)
    background: str | None = None,          # Domain knowledge and constraints
```

```
    config: GEPAConfig | None = None,       # Engine, reflection, tracking settings
) -> GEPAResult:
    """
    Call with either (seed_candidate+evaluator) or (evaluator+objective)
    """
```

Notice what's *absent*: no mutation prompts, no task-specific instruction templates, no island configurations, no EVOLVE-BLOCK markers (all common in prior LLM-evolution frameworks). You declare the **what** (your artifact, your evaluator, and any domain knowledge as `background`) and `optimize_anything` handles the **how**: prompt construction, reflection, candidate selection, and search strategy. This declarative design, inspired by DSPy's principle of *programming not prompting*, means the same API call works whether you're optimizing a CUDA kernel, a cloud scheduling policy, or an agent architecture.

Let's Take It for a Spin

Let's use `optimize_anything` to optimize SVG source code depicting "a pelican riding a bicycle" starting from a blank white canvas. The evaluator renders the SVG as a PNG, asks a VLM to score it against visual criteria, and passes the rendered image back as ASI so the proposer can literally *see* what it's improving. Here's the zero-shot baseline from Claude Opus 4.6 versus the best optimized result after exploring 20 candidates:



Zero-shot attempt from Claude Opus 4.6

Best candidate (score: 0.817)

The optimizer added background elements, improved anatomy, increased the sophistication of all visual elements, and refined the composition — all through LLM reflection on rendered image feedback.

Notably, we optimize the SVG code itself, not a prompt that generates SVG. Here's the code.

First, we define our evaluator and the visual aspects we'd like it to grade for:

Defining the evaluator

```
from gepa import Image
from demo_utils import render_image, get_vlm_score_feedback

GOAL = "a pelican riding a bicycle"
VLM = "vertex_ai/gemini-3-flash-preview"

def evaluate(candidate, example):
    """Render SVG → image, score with a VLM, return (score, side_info)."""
    image = render_image(candidate["svg_code"]) # via cairosvg
    score, feedback = get_vlm_score_feedback(VLM, image, example["criteria"]) # simple
regex parser

    return score, {
        "RenderedSVG": Image(base64_data=image, media_type="image/png"),
        "Feedback": feedback,
    }

VISUAL_ASPECTS = [
    # 6 visual aspects → Pareto-efficient selection
    {"id": "overall",     "criteria": f"Rate overall quality of this SVG ({GOAL}). SCORE:
X/10"},
```

```
    {"id": "anatomy",     "criteria": "Rate pelican accuracy: beak, pouch, plumage.
SCORE: X/10"},
    {"id": "bicycle",     "criteria": "Rate bicycle: wheels, frame, handlebars, pedals.
SCORE: X/10"},
    {"id": "composition", "criteria": "Rate how convincingly the pelican rides the
bicycle. SCORE: X/10"},
    {"id": "visual",      "criteria": "Rate visual appeal, scenery, and color usage.
SCORE: X/10"},
    {"id": "craft",       "criteria": "Rate SVG technical quality: shapes, layering.
SCORE: X/10"},
]
```

Then, we put it all together and run `optimize_anything`:

Running optimize_anything

```
from gepa.optimize_anything import (
    optimize_anything, GEPAConfig, EngineConfig, ReflectionConfig,
)

result = optimize_anything(
    seed_candidate={"svg_code": open("seed.svg").read()}, # a plain white canvas
    evaluator=evaluate,
    dataset=VISUAL_ASPECTS,
    objective=f"Optimize SVG code to illustrate '{GOAL}'. Output ONLY valid SVG.",
    config=GEPAConfig(
        engine=EngineConfig(max_metric_calls=150),
```

```
        reflection=ReflectionConfig(reflection_lm=VLM),
    ),
)
print(result.best_candidate)
```

A few things to note:

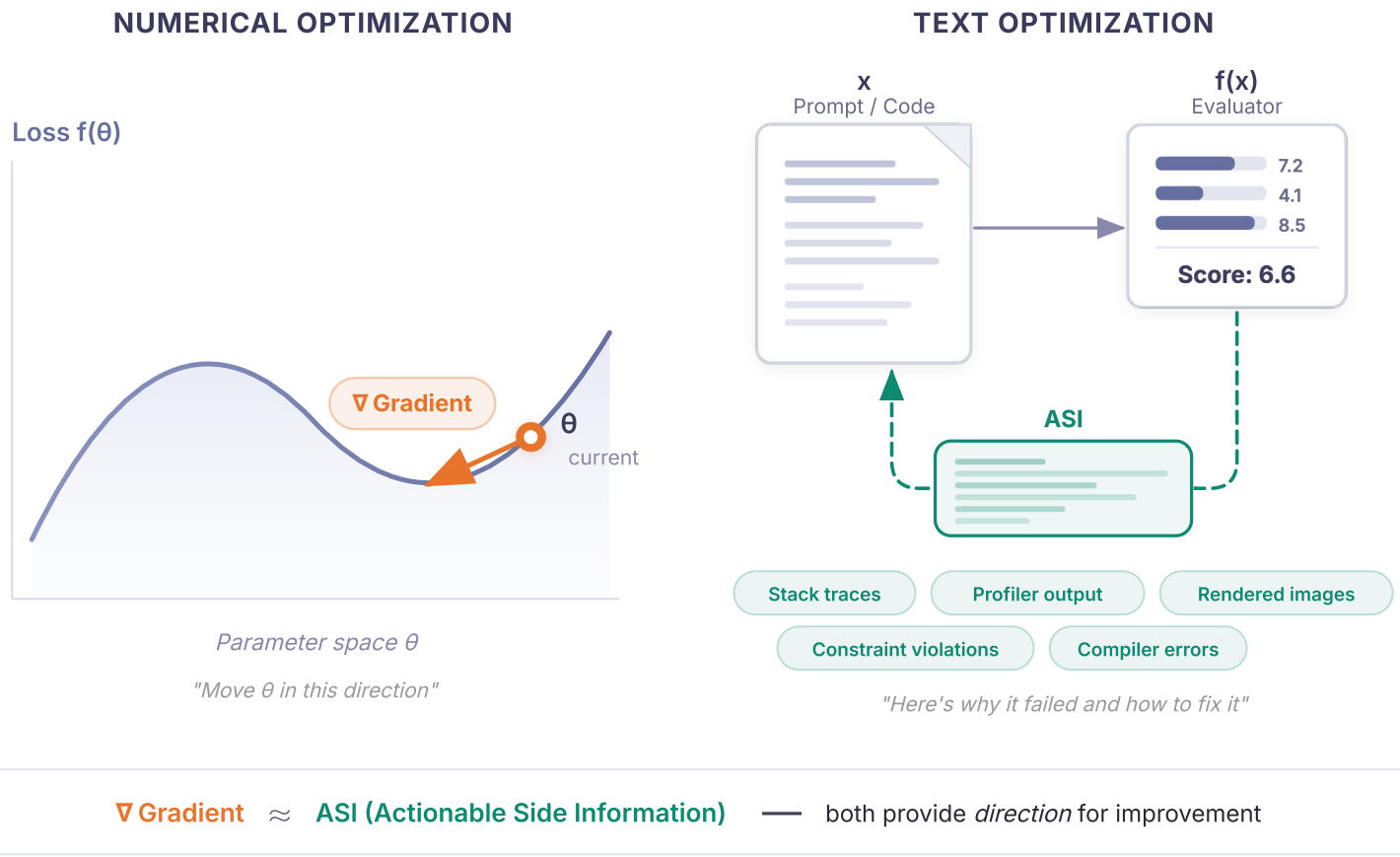
- The `dataset` contains 6 evaluation aspects. GEPA calls the evaluator once per aspect per candidate, tracking scores individually. This enables Pareto-efficient selection: a candidate that excels at bicycle structure but struggles with pelican anatomy is preserved on the frontier, not discarded because its overall average score is smaller.
- Our desired visual aspects are defined in concise natural language. We avoid the need for detailed rubrics and simply rely on the VLM's judgment for scoring.
- `reflection_minibatch_size=2` (the default) means each reflection step shows the LLM feedback from just 2 of the 6 aspects. Over multiple iterations, all aspects get attention, but each reflection is focused and targeted.
- The **rendered image** is passed as ASI via `Image(base64_data=...)`, giving the VLM proposer visual feedback on its own output. The VLM evaluator never sees the SVG code, only the rendered image. The proposer sees both the feedback and the source SVG, and proposes targeted improvements.

How It Works

Classical optimization methods reduce all diagnostic context to a single scalar. They know *that* a candidate failed, but not *why*. You can't show a Bayesian optimizer the stack trace that pinpoints the bug. Recent LLM-evolution frameworks changed this by feeding execution results and textual feedback into LLM proposers. However, the "evolutionary" framing these frameworks inherit suggests a blind process — mutate, evaluate, select, repeat. But when an LLM reads a compiler error, diagnoses a logic bug, and proposes a targeted fix, that's not natural selection, it's an engineer iterating on a prototype. `optimize_anything` leans into this with two key ingredients: **diagnostic feedback as a first-class API concept** and **Pareto-efficient search**.

Actionable Side Information (ASI)

`optimize_anything` makes diagnostic feedback a **first-class part of the evaluator contract**. Prior frameworks expose feedback through framework-specific mechanisms; ASI provides a uniform interface that makes it trivial to surface any diagnostic the evaluator can produce, including modalities no prior framework supports, such as rendered images that let a VLM visually inspect its own output. In the pelican demo, the evaluator passed the rendered SVG back as an image so the proposer could literally *see* what it was improving. During a dedicated reflection step, the proposer reasons over this signal to diagnose failures and propose targeted fixes.



ASI is the text-optimization analogue of the gradient. Where gradients tell a numerical optimizer which direction to move, ASI tells an LLM proposer why a candidate failed and how to fix it.

Pareto-Efficient Search

Even when optimizing a single objective, evaluating candidates across multiple aspects or examples produces richer signal. The naive approach collapses that signal into one average score and always improves the top candidate. This stalls fast: averaging hides which aspects are strong and which are weak, and the proposer tries to improve everything at once instead of focusing.

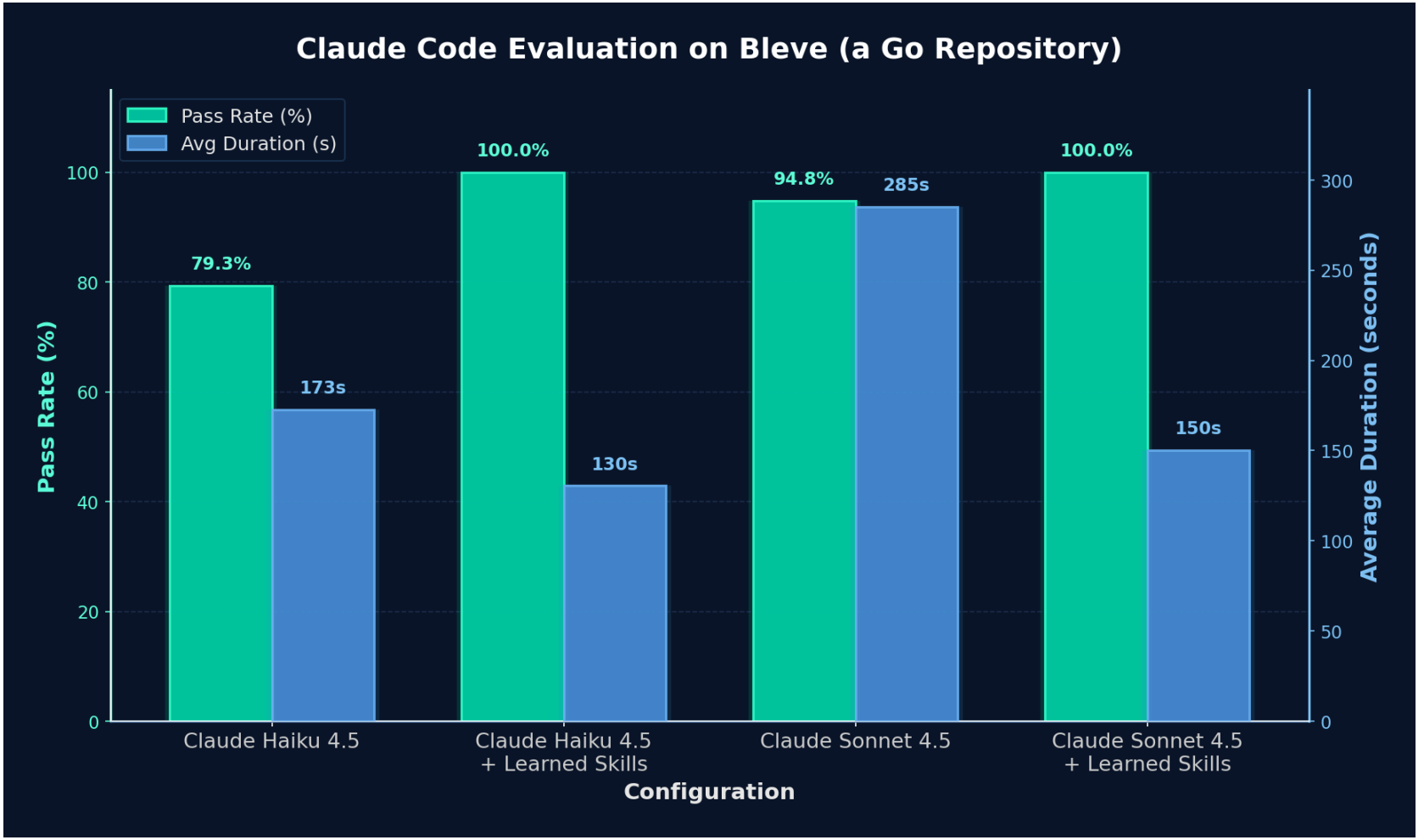
`optimize_anything` does two things differently. First, it tracks scores per task (expressed in `dataset` or `valset`) or metric (expressed in returned score from evaluator and `scores` field in ASI) individually and maintains a **Pareto frontier**: any candidate that is the best at *something* survives, even if its average is suboptimal. Second, each reflection step shows the proposer a minibatch of just 2–3 examples or metrics instead of all of them. The proposer makes focused, targeted improvements on that subset, and the Pareto frontier ensures these specialized gains are preserved across iterations rather than averaged away. Over iterations, the frontier accumulates complementary strengths, and the best candidates combine them. The same mechanism powers multi-task search: when optimizing across a batch of related problems, the frontier preserves candidates that excel on different tasks, and strategies discovered for one problem transfer to others — which is why multi-task mode outperforms dedicated single-task optimization on CUDA kernel generation.

Results

`optimize_anything` learns repository-specific skills that push coding agents to near-perfect accuracy, discovers novel cloud scheduling algorithms that cut costs by 40%, evolves a 10-line agent stub into a 300+ line system that nearly triples its test accuracy on ARC-AGI, boosts GPT's math reasoning via prompt optimization, generates fast CUDA kernels, beats AlphaEvolve's solution at circle packing, matches Optuna (a mature numerical optimizer) by generating custom solver code from scratch, and models a 3D unicorn from no seed code. We test across eight domains spanning search, batch optimization, and generalization. Each section below walks through the examples and links to full, runnable code.

1. Optimize Agent Skills: Near-Perfect Claude Code Accuracy, 47% Faster

Mode: Generalization. Skills (natural-language instructions and best practices for working with a specific codebase) are text artifacts too. `optimize_anything` can optimize them: the evaluator runs a coding agent on real tasks from the repository and scores whether it resolves them; the optimized skills must generalize to unseen tasks.



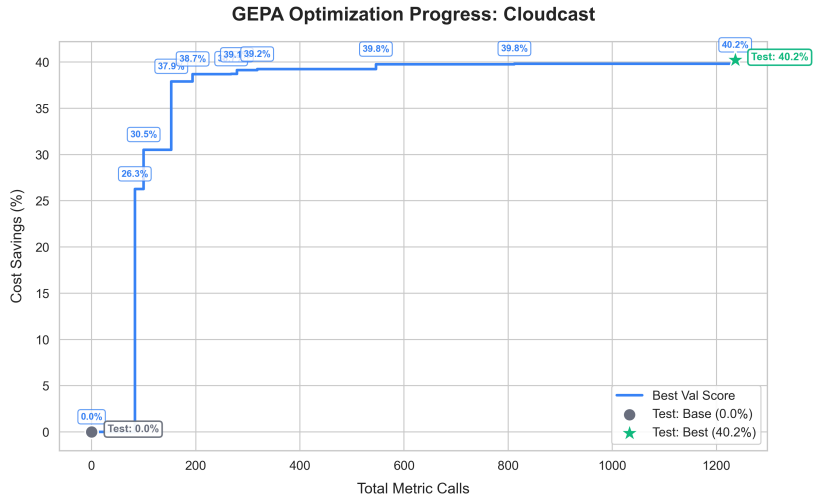
Claude Code on Bleve: optimized skills boost Haiku 4.5 pass rate from 79.3% to 100% and Sonnet 4.5 from 94.8% to 100%, while reducing resolve duration by 47%.

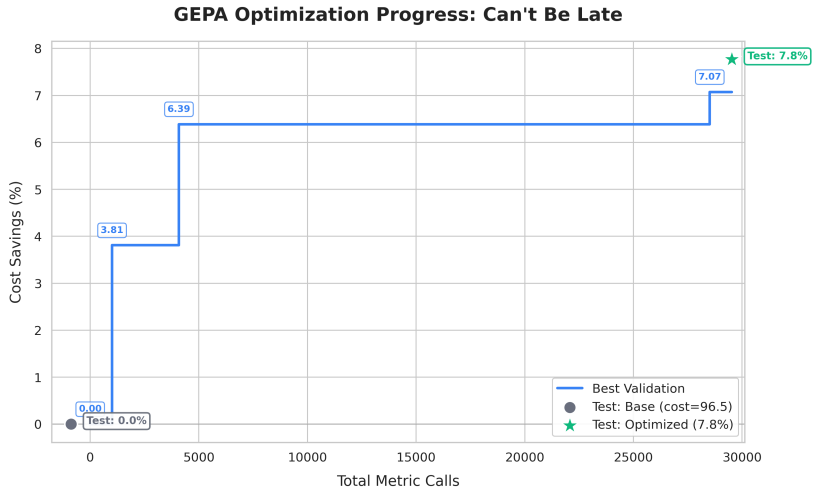
The results are striking: GEPA-optimized skills boost resolve rates from 24% to **93%** on one repository and from 55% to **82%** on another, and transfer directly to Claude Code, pushing it to near-perfect pass rates while cutting resolution time by **47%**.

Key result: `optimize_anything` learns repository-specific skills that dramatically improve coding agent performance and transfer across models. Read the full post →

2. Discover Cloud Algorithms That Cut Costs up to 40%

Mode: Generalization. We optimize cloud infrastructure algorithms: **CloudCast** discovers broadcast routing strategies for multi-cloud data transfer (minimizing egress cost), and **Can't Be Late** learns scheduling policies that decide when to use cheap-but-preemptible SPOT instances versus reliable ON_DEMAND instances to complete tasks before deadlines.





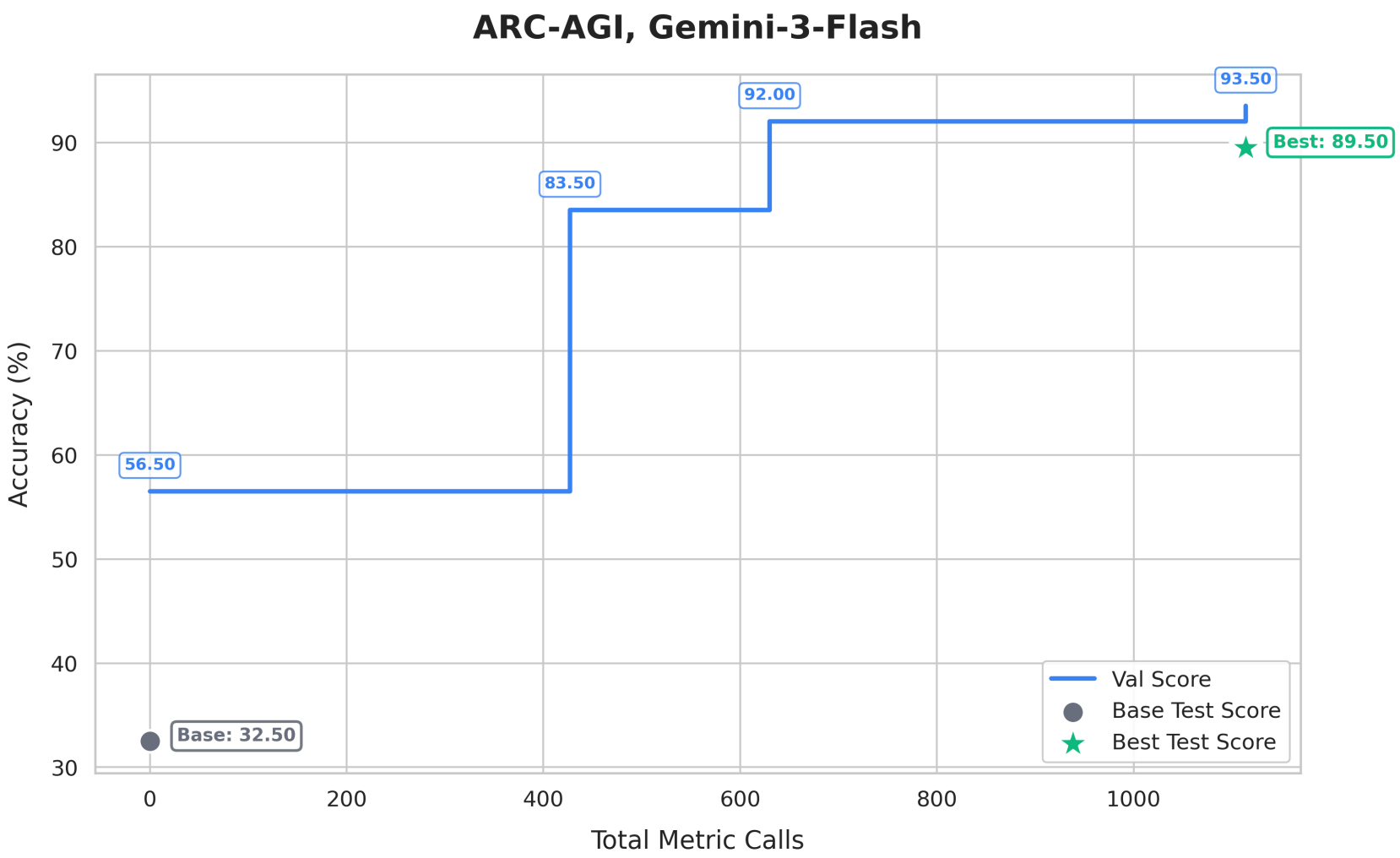
CloudCast (40.2% cost savings): *Optimizes from baseline Dijkstra routing to a provider-aware Steiner tree algorithm with Pareto-frontier candidate selection.*

Can't Be Late (7.8% cost savings): *Optimizes from a simple deadline-check heuristic to an adaptive scheduling strategy that tracks spot availability patterns and computes break-even switching costs.*

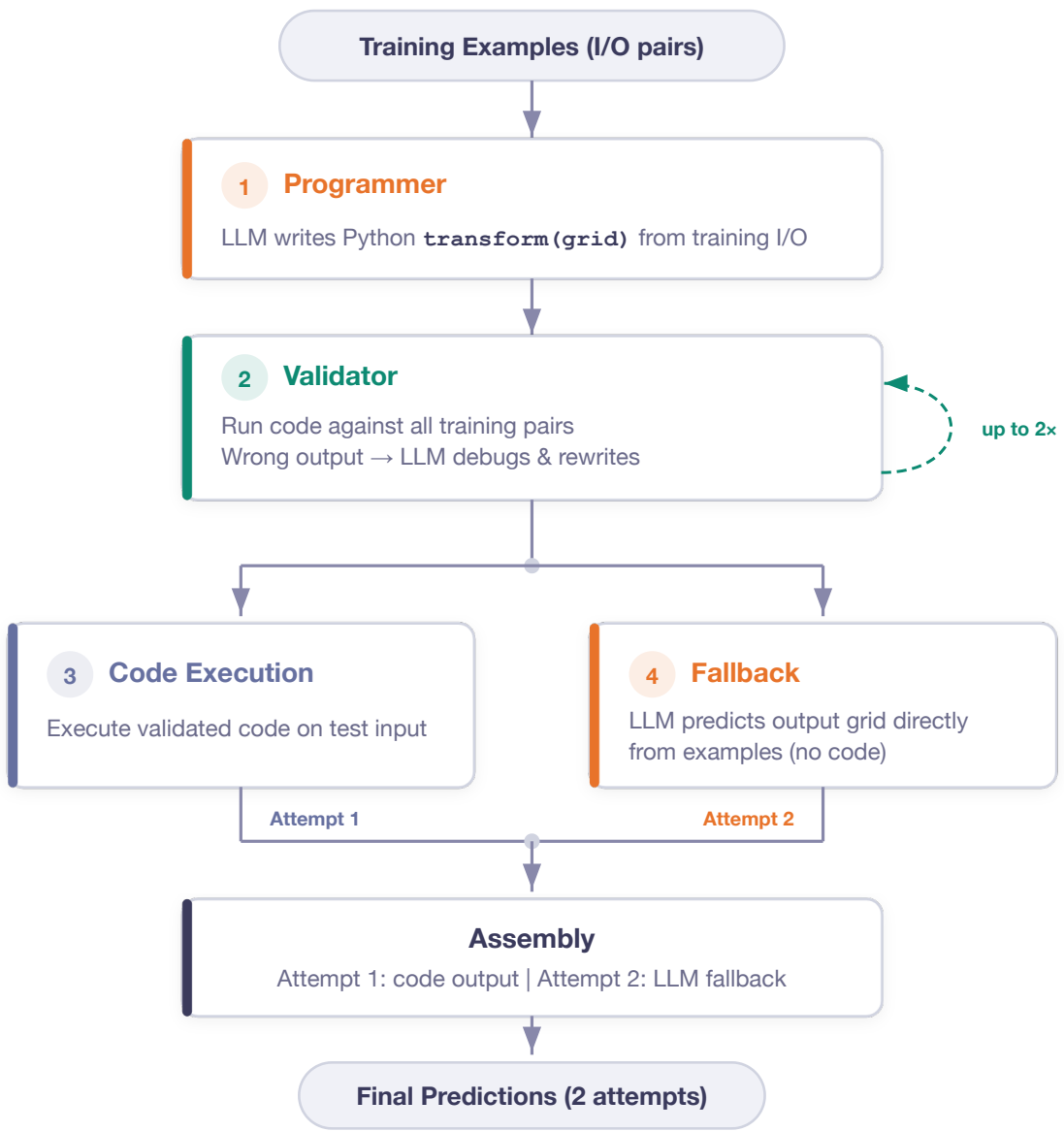
Key result: `optimize_anything` discovers state-of-the-art algorithms for both problems (40.2% cost savings on CloudCast and 7.8% cost savings on Can't Be Late), topping the ADRS leaderboard (outperforming OpenEvolve, ShinkaEvolve, and expert-designed heuristics). CloudCast code → | Can't Be Late code →

3. Nearly Triple Gemini-Flash's ARC-AGI Accuracy via Agent Architecture Evolution

Mode: Generalization. This is the most ambitious application. Rather than optimizing a prompt, we optimize the **entire agent system**: code, sub-agent architecture, control flow, helper functions, and prompts are all treated as a single text artifact. The seed is a 10-line naive agent; GEPA evolves it into a 300+ line system with rule induction, code verification, iterative refinement, and structured fallbacks. It nearly triples Gemini Flash's ARC-AGI accuracy at just twice the cost per task.



ARC-AGI agent evolution: from a naive agent (32.5% test) to a sophisticated 300+ line system (89.5% test) with Gemini 3 Flash.

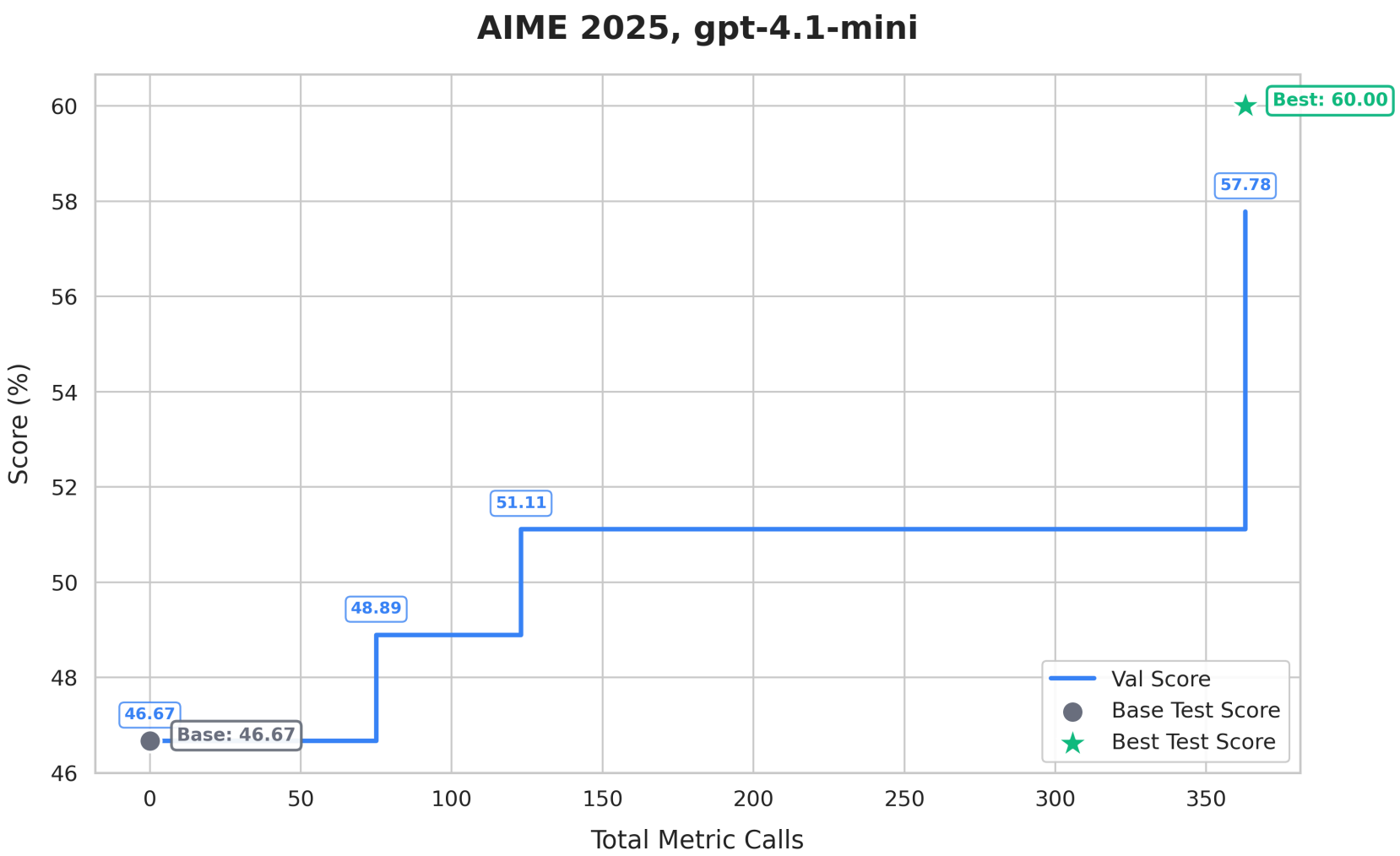

Training Examples (I/O pairs)
1
Programmer
LLM writes Python transform(grid) from training I/O
2
Validator
Run code against all training pairs
Wrong output → LLM debugs & rewrites
up to 2×
3
Code Execution
Execute validated code on test input
4
Fallback
LLM predicts output grid directly from examples (no code)
Attempt 1
Attempt 2
Assembly
Attempt 1: code output | Attempt 2: LLM fallback
Final Predictions (2 attempts)

The optimized ARC-AGI agent architecture: code generation, iterative validation, and two-attempt prediction (code + direct LLM)

Key result: Using the same underlying model (Gemini 3 Flash), `optimize_anything` improves ARC-AGI v1 public test accuracy from 32.5% to **89.5%** by evolving the entire agent architecture, achieving gains that typically require significant manual iteration. Full code →

4. Improve GPT's Math Accuracy via Prompt Optimization (AIME)

Mode: Generalization. We optimize a system prompt for gpt-4.1-mini by *training* on AIME 2022–2024 math competition problems and *testing* on AIME 2025. GEPA sets the state-of-the-art for prompt optimization.

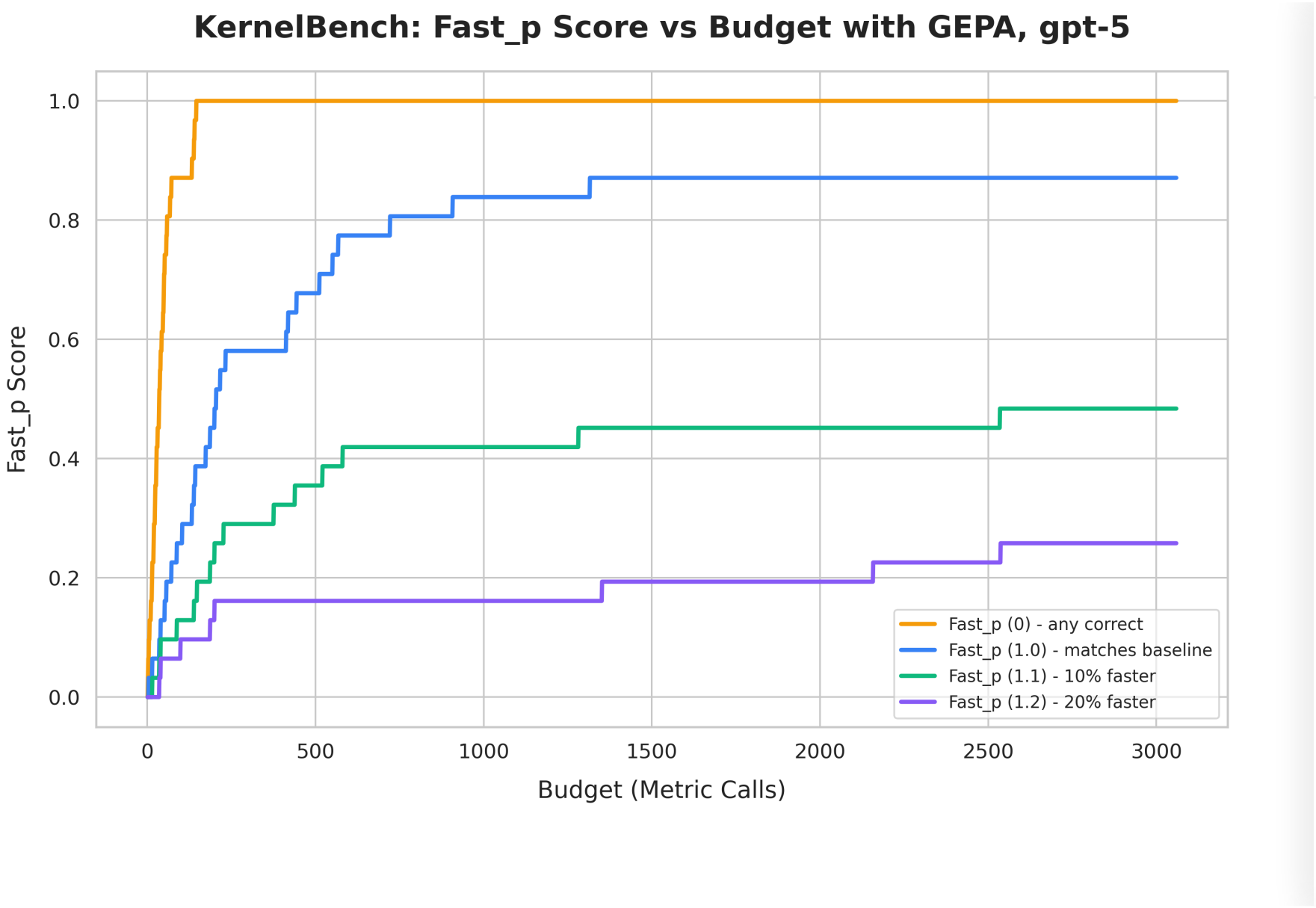


AIME 2025 prompt optimization: gpt-4.1-mini accuracy improves from 46.67% to 60.00% through prompt refinement alone.

Key result: Pure prompt optimization improves gpt-4.1-mini from 46.67% to **60.00%** on AIME 2025, a 13.3 percentage point gain from changing only the system prompt. Full code →

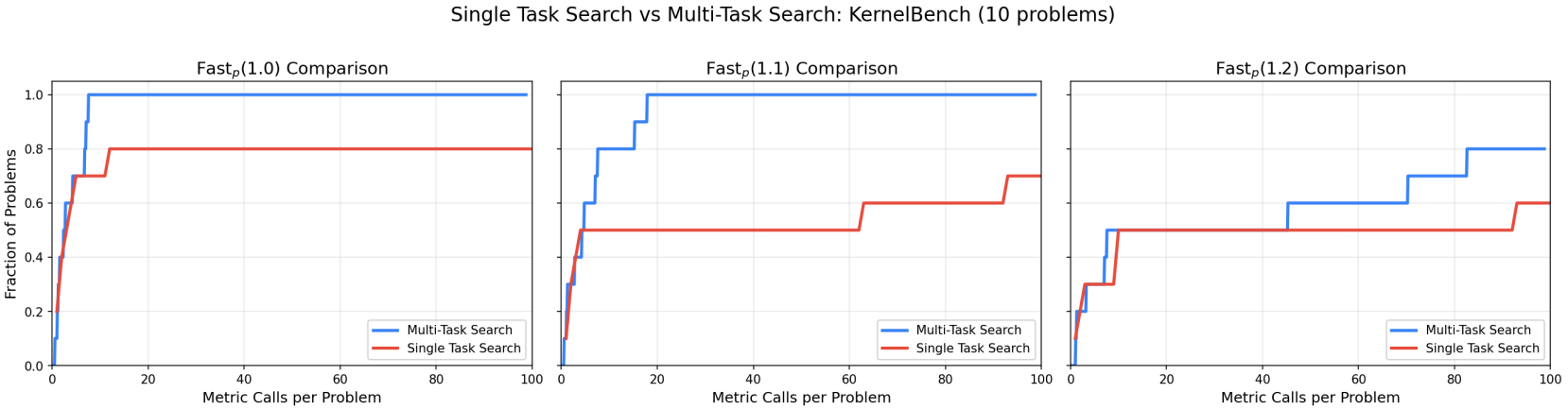
5. Accelerate PyTorch with Custom CUDA Kernels

Mode: Multi-Task Search. We generate fast CUDA kernels for multiple reference PyTorch operations from KernelBench, evaluated on a V100 32 GB GPU. Under the hood, GEPA evolves the prompt that drives kernel generation, so improvements discovered for one problem transfer to others automatically.


KernelBench: Fast_p Score vs Budget with GEPA, gpt-5
Fast_p Score
1.0
0.8
0.6
0.4
0.2
0.0
0
500
1000
1500
2000
2500
3000
Budget (Metric Calls)
Fast_p (0) - any correct
Fast_p (1.0) - matches baseline
Fast_p (1.1) - 10% faster
Fast_p (1.2) - 20% faster

KernelBench results with GEPA (gpt-5 as proposer). 87% of generated kernels match or beat baseline performance; 25% are 20%+ faster. We use 31 of the 35 hand-curated problems from the KernelBench authors.[1]

To gauge the effectiveness of cross-task learning, we take the 10 problems where multi-task mode performed best and re-optimize each from scratch in single-task mode to see whether a dedicated single-task run can beat the multi-task result. The graph below shows that a multi-task mode converges faster and solves more problems across all speedup thresholds.

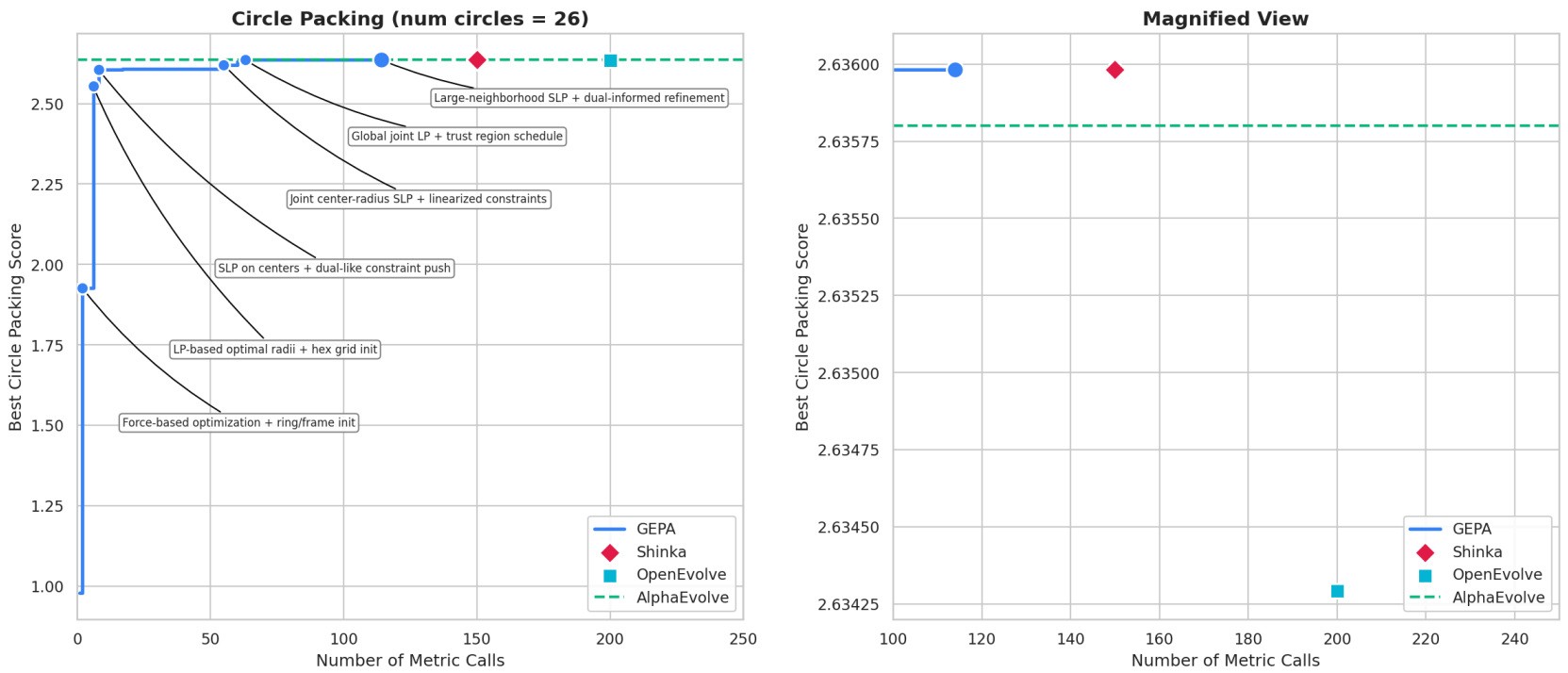


Single-task vs multi-task mode on 10 KernelBench problems.

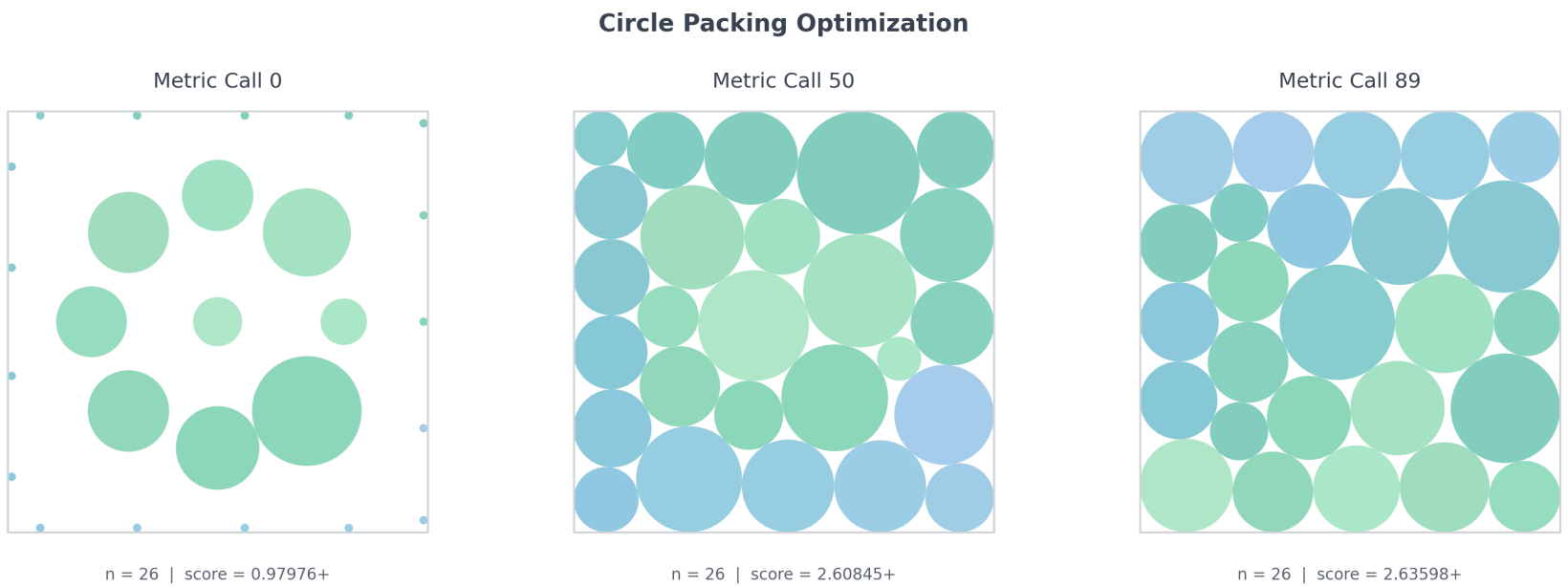
Key result: 87% of GEPA-generated kernels match or beat the baseline, with 25% achieving 20%+ speedups. Multi-task mode outperforms dedicated single-task search modes, suggesting the efficiency of cross-task learning. Full code →

6. Outperform AlphaEvolve's solution at Circle Packing

Mode: Single-Task Search. Pack n=26 circles to maximize the sum of their radii within a unit square. GEPA optimizes the packing algorithm code, using execution results and geometric diagnostics as ASI.



GEPA outperforms AlphaEvolve, ShinkaEvolve, and OpenEvolve's solutions on circle packing (n=26), reaching a higher score with fewer evaluations.

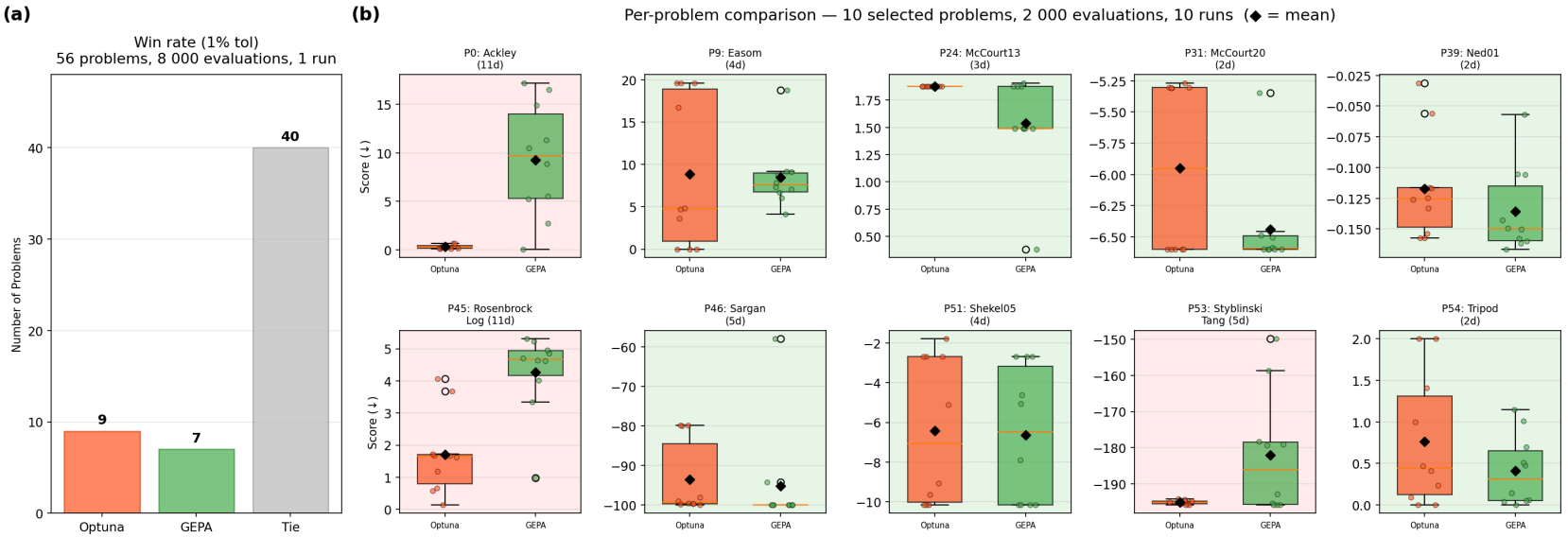


Visual progression of the circle packing optimization: from an initial naive arrangement to a near-optimal packing.

Key result: GEPA outperforms prior LLM-evolution frameworks (AlphaEvolve/ShinkaEvolve/OpenEvolve), reaching a score of 2.63598+. Full code →

7. Match or Outperform Optuna at Blackbox Mathematical Optimization

Mode: Single-Task Search. Given a blackbox objective function, `optimize_anything` discovers an optimization algorithm tailored to it and matches Optuna, the industry-standard blackbox optimizer, across the 56-problem EvalSet benchmark.



GEPA's `optimize_anything` matches Optuna, the industry-standard blackbox optimizer, on the EvalSet benchmark. (a) Across all 56 EvalSet problems (budget of 8,000 evaluations each), GEPA ties Optuna on 40, wins 7, and loses 9. (b) On 10 selected problems where Optuna struggles (budget of 2,000 evaluations each), GEPA finds better solutions on 7 out of 10.

On the 56-problem evalset benchmark with large budgets, GEPA and Optuna tie on most problems. But on the hardest problems with lower budgets where Optuna struggles, an interesting pattern emerges: Optuna's fixed TPE-CMA-ES pipeline fails in predictable, structural ways. On McCourt13, all 10 independent Optuna runs converge to the same local minimum because TPE's independent per-dimension sampling always falls into the dominant trap basin. On Tripod, CMA-ES assumes a smooth, unimodal landscape, but the objective is piecewise-linear with hard discontinuities, so it converges to the wrong basin and cannot escape.

GEPA tailors the solver to each problem by learning from accumulated evaluation history. For boundary optima, it discovers L-BFGS-B, a box-constrained optimizer that naturally sticks to boundaries. For deceptive traps, it designs multi-start search from diverse starting points, escaping basins that trap single-trajectory methods. While Optuna tunes parameters within a fixed algorithm, GEPA learns to optimize the algorithm itself on the fly.

Key result: `optimize_anything` matches the performance of Optuna, a mature numerical optimizer, by optimizing a blackbox search program tailored to each problem. Full code →

8. From Vague Idea to 3D Unicorn — No Seed Required

Mode: Multi-Task Search (seedless). Every example so far starts from a hand-written seed: a blank SVG, a naive agent stub, a baseline algorithm. But what if you don't even know where to begin? You know *what* you

want — a 3D unicorn — and you can articulate *what good looks like* (anatomical accuracy, mesh quality, visual appeal), but you have no idea how to wire up build123d geometry, STL export, and pyrender camera orbits into a working script.

This is exactly what seedless mode (`seed_candidate=None`) is for. Instead of providing a starting artifact, you describe the objective and the technical context as natural language, and GEPA's reflection LM bootstraps the first candidate itself:

```
result = optimize_anything(
    seed_candidate=None,  # no starting code — the LM writes the first draft
    evaluator=evaluate_3d_render,
    dataset=VISUAL_ASPECTS,   # 4 aspects: quality, anatomy, mesh, appeal
    objective="Optimize a Python program (build123d + pyrender) to generate a 3D
unicorn.",
    background=(
        "The candidate is a complete Python script that produces multi-view PNG
renderings. "
        "Use build123d for CSG geometry, export to STL, render with pyrender. "
        "Known working imports: numpy, pyrender, trimesh, build123d (Box, Cylinder, Cone,
...)."
    ),
)
```

The evaluator runs each candidate as a subprocess, collects the rendered PNGs, and asks a VLM to score them — passing the images back as ASI so the proposer can *see* what its code produces. Here's Claude Opus 4.6's zero-shot attempt versus the GEPA-optimized result:

Zero-shot from Claude Opus 4.6

GEPA-optimized (seedless using Claude Opus 4.6)

The zero-shot model produces a recognizable but crude unicorn — blocky torso, piston-like legs, a horn on a box head. GEPA iteratively refines the geometry, improving proportions, adding anatomical detail, and even adds a swirl around the horn, all without any human-written seed code to start from.

The seedless mode is particularly useful for tasks where the solution space is large and unfamiliar such as creative or exploratory tasks. You bring the evaluation criteria (your *taste*) and the optimizer handles everything else. Full code →

Conclusion & Getting Started

`optimize_anything` has a simple premise: if your artifact is text and its performance can be measured, you can optimize it. The API is minimal, requiring only a seed (or task description), an evaluator, and optionally a dataset. The results span algorithmic discovery, kernel generation, systems research, prompt tuning, agent architecture search, blackbox optimization, and coding agent skill learning.

The key ideas: (1) **three unified modes** (single-task search, multi-task search, and generalization) under one declarative API; (2) **Actionable Side Information (ASI)** as a first-class API concept that turns blind mutation into targeted, diagnostic-driven engineering; (3) **Pareto-efficient search** across metrics and examples that outperforms naive all-at-once optimization.

By design, `optimize_anything` is a general frontend for text optimization. It is currently powered by GEPA as the optimization backend, but the API is backend-agnostic: as new optimization strategies emerge with increasingly powerful models, they can be plugged in without changing any user code. Our goal is for `optimize_anything` to always dispatch to the best available optimizer for your problem. We welcome community contributions of new optimization backends, evaluators, and case studies.

Get started:

```
pip install gepa
```

```
import gepa.optimize_anything as oa

result = oa.optimize_anything(
    seed_candidate="<your artifact>",
    evaluator=your_evaluator,
)
```

- Documentation
- GitHub
- Discord
- Twitter

- Slack

Appendix: Detailed Code Walkthroughs for each Case Study

Cloud Broadcast Routing (CloudCast)

Generalization mode for cloud infrastructure optimization. Optimizes a Python program that implements a broadcast routing strategy, evaluated against real-world cloud simulations. GEPA discovers algorithms that generalize across diverse network configurations.

CloudCast — broadcast routing for multi-cloud data transfer.

Artifact being evolved: a Python `search_algorithm` function (the full routing algorithm code) that receives a network graph of cloud regions and must return a `BroadCastTopology` — a set of routing paths from a source to multiple destinations across AWS, GCP, and Azure. The network is a directed graph where nodes are cloud regions and edges carry `cost` ($/GB) and `throughput` (Gbps) attributes. GEPA evolves the entire algorithm, including data structures it uses.

Seed Candidate — A baseline Dijkstra shortest-path router. It finds the single cheapest path to each destination and sends all data partitions along the same route:

```
SEED_PROGRAM = """
import networkx as nx
from typing import Dict, List

class BroadCastTopology:
    def __init__(self, src, dsts, num_partitions=4, paths=None):
        self.src = src
        self.dsts = dsts
        self.num_partitions = num_partitions
        self.paths = paths or {dst: {str(i): None for i in range(num_partitions)}
for dst in dsts}

    def set_num_partitions(self, num_partitions):
        self.num_partitions = num_partitions

    def set_dst_partition_paths(self, dst, partition, paths):
        self.paths[dst][str(partition)] = paths

    def append_dst_partition_path(self, dst, partition, path):
        partition = str(partition)
        if self.paths[dst][partition] is None:
            self.paths[dst][partition] = []
        self.paths[dst][partition].append(path)

def search_algorithm(src, dsts, G, num_partitions):
    \"\"\"Baseline: Dijkstra shortest-cost path, same route for all
partitions.\"\"\"
    h = G.copy()
```

```
    h.remove_edges_from(list(h.in_edges(src)) + list(nx.selfloop_edges(h)))
    bc_topology = BroadCastTopology(src, dsts, num_partitions)

    for dst in dsts:
        path = nx.dijkstra_path(h, src, dst, weight="cost")
        for i in range(len(path) - 1):
            s, t = path[i], path[i + 1]
            for j in range(num_partitions):
                bc_topology.append_dst_partition_path(dst, j, [s, t, G[s][t]])

    return bc_topology
"""
```

Evaluator — The evaluator runs the candidate's `search_algorithm` through a broadcast simulator that models real cloud egress costs and bandwidth constraints. `get_program_path` caches the candidate to a temp file (keyed by content, so repeated calls are free). `run_evaluation` loads it as a Python module, executes it on the network graph, and runs the simulator to compute cost and transfer time.

ASI (Actionable Side Information): The side information includes (1) per-destination route breakdowns with egress cost and transfer time, (2) cost decomposition into egress vs. instance components, and (3) bottleneck destination identification. This tells the LLM proposer *where* the

algorithm is wasting money (e.g., expensive cross-provider hops) and *which* destinations are slowest, guiding targeted improvements.

```
from utils.simulation import (
    FAILED_SCORE,
    get_program_path, syntax_is_valid, syntax_failure_info,
    run_evaluation, evaluation_failure_info, evaluation_success_info,
)

def evaluate(candidate: dict, example: dict, **kwargs):
    program_path = get_program_path(candidate["program"])
    if not syntax_is_valid(program_path):
        return FAILED_SCORE, syntax_failure_info(example)
    success, cost, transfer_time, error, details = run_evaluation(
        program_path, example["config_file"], example["num_vms"]
    )
    if not success:
        return FAILED_SCORE, evaluation_failure_info(error, example)
    score = 1.0 / (1.0 + cost)
    return score, evaluation_success_info(score, cost, transfer_time, example,
details)
```

Optimizer — Generalization mode with 5 multi-cloud broadcast configurations as both the training and validation set (intra-AWS, intra-Azure, intra-GCP, and two cross-cloud scenarios). The evolved

algorithm must generalize across intra- and inter-provider network topologies.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig,
ReflectionConfig
from utils.dataset import load_config_dataset
from utils.lm import make_reflection_lm

# 5 multi-cloud broadcast configs: intra_aws, intra_azure, intra_gcp, inter_agz,
inter_gaz2
dataset = load_config_dataset()

result = optimize_anything(
    seed_candidate={"program": SEED_PROGRAM},
    evaluator=evaluate,
    dataset=dataset,
    valset=dataset,
    objective="Optimize a broadcast routing algorithm for multi-cloud data transfer.
"
              "Minimize total cost (egress fees + instance costs) while maintaining
"
              "good transfer times.",
    background="Nodes are cloud regions (e.g. 'aws:us-east-1', 'gcp:europe-west1-
a'). "
               "Edges have 'cost' ($/GB egress) and 'throughput' (Gbps). Data is
split "
               "into num_partitions chunks routable independently. Total cost =
egress "
```

```
                "cost + instance runtime cost. Intra-provider links are typically
cheaper.",
    config=GEPAConfig(
        engine=EngineConfig(max_metric_calls=100),
        reflection=ReflectionConfig(reflection_lm=make_reflection_lm("gemini-3-pro-
preview")),
    ),
)
```

Optimized artifact — The evolved algorithm achieves **40.2% cost savings** over the Dijkstra baseline. Starting from a simple single-path router, GEPA discovered a sophisticated provider-aware Steiner tree algorithm with Pareto-frontier candidate selection and greedy partition allocation. Key innovations: (1) provider-penalty weighting to prefer cheap intra-provider links, (2) diverse candidate generation via multiple Steiner tree strategies, (3) Pareto filtering on cost vs. time, and (4) incremental partition assignment that models bandwidth contention.

```
import networkx as nx
import random
import math
from typing import Dict, List, Set, Tuple, Any
from collections import defaultdict
```

```
class SingleDstPath(Dict):
    partition: int
    edges: List[List]  # [[src, dst, edge data]]

class BroadCastTopology:
    def __init__(self, src: str, dsts: List[str], num_partitions: int = 4,
paths: Dict[str, 'SingleDstPath'] = None):
        self.src = src
        self.dsts = dsts
        self.num_partitions = num_partitions
        if paths is not None:
```

Cloud Spot Scheduling (Can't Be Late)

Generalization mode for cloud infrastructure optimization. Optimizes a Python program that implements a scheduling strategy, evaluated against real-world spot-availability traces. GEPA discovers scheduling policies that generalize across diverse job configurations and spot-availability patterns.

Can't Be Late — cloud scheduling with SPOT vs ON_DEMAND instances.

Artifact being evolved: a Python scheduling policy (the `_step` method of a `Strategy` class) that is called at each time step and must return one of three actions: `ClusterType.SPOT` (~0.30/hr, cheap but preemptible), `ClusterType.ON_DEMAND` (~1.00/hr, reliable), or `ClusterType.NONE` (wait, no cost). The policy has access to remaining task time, deadline, restart overhead, and spot availability. GEPA evolves the entire strategy class, including any state variables it tracks.

Seed Candidate — A simple greedy heuristic: use ON_DEMAND only when the deadline is imminent, otherwise prefer SPOT when available, and wait when it's not:

```
SEED_PROGRAM = """
import math
from sky_spot.strategies.strategy import Strategy
from sky_spot.utils import ClusterType

class EvolveSingleRegionStrategy(Strategy):
    NAME = 'evolve_single_region'

    def __init__(self, args):
        super().__init__(args)

    def reset(self, env, task):
        super().reset(env, task)

    def _step(self, last_cluster_type: ClusterType, has_spot: bool) -> ClusterType:
        env = self.env

        # Task completion check
        remaining_task_time = self.task_duration - sum(self.task_done_time)
        if remaining_task_time <= 1e-3:
            return ClusterType.NONE

        # Calculate remaining time until deadline
        remaining_time = self.deadline - env.elapsed_seconds

        # If running out of time, use ON_DEMAND to guarantee completion
        if remaining_task_time + self.restart_overhead >= remaining_time:
            return ClusterType.ON_DEMAND
```

```
        # Greedy: use SPOT if available, otherwise wait
        if has_spot:
            return ClusterType.SPOT
        else:
            return ClusterType.NONE

    @classmethod
    def _from_args(cls, parser):
        args, _ = parser.parse_known_args()
        return cls(args)
"""
```

Evaluator — The evaluator runs the candidate strategy through a scheduling simulator driven by real spot-availability traces. `get_program_path` caches the candidate to a temp file (keyed by content, so repeated calls are free). `run_simulation` handles subprocess execution of the simulator and cost extraction. Each trace is tested across multiple job configurations (varying task duration, deadline tightness, and restart overhead).

ASI (Actionable Side Information): The side information includes (1) the full spot-availability pattern for the trace (e.g., `"0.0-10.0:S | 10.0-15.0:X"` — spot available for 10h then unavailable), (2) a timeline of the strategy's instance usage decisions (e.g., `"0.0-5.0:S@R0[50%] | 5.0-`

`8.0:OD@R0[100%]"`), and (3) segment counts (SPOT vs ON_DEMAND vs restarts). This lets the LLM proposer see *exactly when* the strategy made suboptimal decisions — for instance, switching to expensive ON_DEMAND too early when spot was about to become available again.

```
from utils.simulation import (
    FAILED_SCORE,
    get_program_path, syntax_is_valid, syntax_failure_info,
    run_simulation, simulation_failure_info, simulation_success_info,
)

def evaluate(candidate: dict, example: dict, **kwargs):
    program_path = get_program_path(candidate["program"])
    if not syntax_is_valid(program_path):
        return FAILED_SCORE, syntax_failure_info(example)
    success, cost, error, details = run_simulation(
        program_path, example["trace_file"], example["config"]
    )
    if not success:
        return FAILED_SCORE, simulation_failure_info(error, example)
    score = -cost
    return score, simulation_success_info(score, example, details)
```

Optimizer — Generalization mode with spot-availability traces split into training and validation sets. Each trace is evaluated across multiple deadline/overhead configurations, so the evolved strategy must handle both tight and relaxed deadlines. `parallel=True` with 128 workers enables fast evaluation across the large trace dataset.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig,
ReflectionConfig
from utils.dataset import load_trace_dataset
from utils.lm import make_reflection_lm

# Load spot-availability traces split into train/val/test
splits = load_trace_dataset()
train_set, val_set = splits["train"], splits["val"]

result = optimize_anything(
    seed_candidate={"program": SEED_PROGRAM},
    evaluator=evaluate,
    dataset=train_set,
    valset=val_set,
    objective="Optimize a cloud scheduling strategy for the 'Can't Be Late' problem.
"
              "Minimize cost while ensuring task completion before deadline.",
    background="ClusterType.SPOT: ~$0.3/hr, cheap but preemptible at any time. "
               "ClusterType.ON_DEMAND: ~$1/hr, guaranteed availability. "
               "ClusterType.NONE: wait with no cost or progress. restart_overhead: "
```

```
                "time penalty when switching instance types. The strategy MUST "
                "ensure deadline completion (hard constraint).",
    config=GEPAConfig(
        engine=EngineConfig(max_metric_calls=100, parallel=True, max_workers=128),

reflection=ReflectionConfig(reflection_lm=make_reflection_lm("anthropic/claude-opus-
4-5-20251101")),
    ),
)
```

Optimized artifact — The evolved strategy achieves **7.8% cost savings** over the baseline. GEPA transformed the simple greedy heuristic into an adaptive strategy with: (1) state tracking for spot unavailability patterns, (2) overhead-aware switching decisions with break-even cost analysis, (3) graduated decision thresholds based on slack ratio (remaining buffer / task time), and (4) multi-factor logic that considers absolute slack, persistent unavailability, and remaining work size.

```
import math
from sky_spot.strategies.strategy import Strategy
from sky_spot.utils import ClusterType

class EvolveSingleRegionStrategy(Strategy):
    NAME = 'evolve_single_region'
```

```
    def __init__(self, args):
        super().__init__(args)
        self.spot_unavailable_count = 0
        self.consecutive_short_spot_windows = 0

    def reset(self, env, task):
        super().reset(env, task)
        self.spot_unavailable_count = 0
        self.consecutive_short_spot_windows = 0

    def step(self, last_cluster_type: ClusterType, has_spot: bool) -> 
```

Agent Architecture Discovery (ARC-AGI)

Here, we tackle ARC-AGI1. This is a Generalization mode where the entire agent architecture is the artifact being optimized. The seed is a 10-line naive agent that makes a single LLM call; GEPA evolves it into a 170+ line multi-stage system with rule induction, code verification, iterative refinement, and structured fallbacks. Test accuracy improves from 32.5% to 89.5% on a public v1 test set.

Candidate — The seed candidate is a minimal 10-line agent that concatenates training examples into a single prompt and asks the LLM to predict outputs directly. It provides a starting template showing the `solve()` API.

```
SEED_AGENT = '''
import json, re
def solve(train_inputs, train_outputs, test_inputs, llm):
    examples = "\\n".join(f"Input: {i}\\nOutput: {o}"
                          for i, o in zip(train_inputs, train_outputs))
    response = llm(f"Solve an ARC AGI puzzle. Training:\\n{examples}\\n"
                   f"Predict outputs as JSON [[...]]:")
    grids = [json.loads(g) for g in re.findall(r"\\[\\[.*?\\]\\]",
             response.replace("\\n", ""))]
    return {"train": grids[:len(train_inputs)],
```

```
            "test": [[g] for g in grids[len(train_inputs):]]}
'''
```

Evaluator — The evaluator sandboxes the agent code, runs it on an ARC-AGI puzzle (providing training input/output pairs and test inputs), and returns rich ASI: training and test scores, execution errors, the actual grid examples, LLM costs, number of calls made, and all model outputs produced inside the agentic architecture. This lets the proposer see not just *what* the agent got wrong, but *how* it reasoned internally.

```
def evaluate(candidate, example):
    result = run_agent(
        agent_code=candidate,
        train_in=example.train_in,
        train_out=example.train_out,
        test_in=example.test_in,
        test_out=example.test_out or None,
        model_id=LLM_MODEL,
        max_llm_calls=MAX_LLM_CALLS,
    )
    llms = result["llms"]
    score = result["test_score"]

    return score, {
```

```
        "score": score,
        "problem_id": example.problem_id,
        "agent_code": candidate,
        "training_score": result["training_score"],
        "test_score": result["test_score"],
        "cost": llm.total_cost,
        "error": result["error"],
        "train_examples": result["train_examples"],
        "test_examples": result["test_examples"],
        **llms.get_traces(),  # number of calls made, LLM costs, model outputs, etc.
    }
```

Optimizer — This is **Generalization** mode with `dataset` for training and `valset` for validation. The agent must generalize to unseen `testset` puzzles, so just memorizing patterns from the training set won't help. `parallel=True` with `max_workers=64` enables massive concurrent evaluation across puzzles. `background` provides domain knowledge about ARC puzzle structure. Gemini 3 Flash is used as both the reflection model and the agent's internal LLM. Note that using a stronger reflection model can find a even more effective artifact in general.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig,
ReflectionConfig
from examples.arc_agi.utils import load_arc_dataset
```

```
train_set, val_set, test_set = load_arc_dataset()

result = optimize_anything(
    seed_candidate={"agent_code": SEED_AGENT},
    evaluator=evaluate,
    dataset=train_set,
    valset=val_set,
    config=GEPAConfig(
        engine=EngineConfig(max_candidate_proposals=40, parallel=True,
                            max_workers=64, cache_evaluation=True),
        reflection=ReflectionConfig(
            reflection_lm="openrouter/google/gemini-3-flash-preview",
        ),
    ),
    objective="Evolve agent code to solve ARC-AGI puzzles. The agent receives "
              "training input/output pairs and must predict test outputs.",
    background="You are allowed to build an agent system with up to 10 LLM calls and
total of $0.8~1.0 LLM cost per problem.",
)
```

Optimized artifact — Starting from a 10-line single-call agent, GEPA discovered a 4-stage pipeline on its own: Analyst (infer the rule), Developer (write a `transform()` function), Validator (run it on all training examples and iteratively debug), and Optimizer (assemble attempts with fallbacks). What's interesting is that the agent learned to generate code, execute it, check the output, and fix bugs in a

loop. It also learned a dual-attempt strategy: try the code path first, fall back to direct LLM prediction if that fails.

```
import json
import re
import numpy as np

def solve(train_inputs, train_outputs, test_inputs, llm):
    """
    ARC-AGI solver using a multi-stage reasoning and execution pipeline:
    1. Analyst: Infers transformation logic and describes it.
    2. Developer: Writes a Python function to implement the logic.
    3. Validator: Tests the code against ALL training examples and iterates if
it fails.
    4. Optimizer: Uses the best-performing code or falls back to direct
prediction via LLM.
    """

    def format_grid(grid):
        return json.dumps(grid)
```

Prompt Optimization (AIME Mathematics)

This is Generalization mode for prompt optimization on AIME 2025 math competition problems. GEPA improves accuracy from 46.67% to 60.00% through prompt refinement alone.

Candidate — The seed is a minimal, generic math instruction. GEPA will evolve this into a detailed problem-solving strategy with specific heuristics for competition-level mathematics.

```
SEED_PROMPT = "Solve the math problem carefully. Break down the steps and provide the final answer as a single number."
```

Evaluator — The evaluator runs the LLM with the candidate prompt on an AIME problem, checks whether the predicted answer matches the ground truth, and returns ASI including the problem statement, written solutions, the model's reasoning chain, and its final answer, helping the LLM generate a more systematic prompt.

```
from examples.aime_math.eval import run_llm, math_metric

def evaluate(candidate: str, example) -> tuple[float, SideInfo]:
```

```
    """Evaluate a candidate on a single example."""
    prediction = run_llm(example, candidate)
    score, feedback = math_metric(example, prediction)

    side_info = {
        "score": score,
        "input": example.input,
        "prompt": candidate,
        "output": prediction.answer,
        "reasoning": getattr(prediction, "reasoning", ""),
        "execution_feedback": feedback,
    }

    return score, side_info
```

Optimizer — This is **Generalization** mode with both a `dataset` (training set) and `valset` (validation set), so the optimized prompt must generalize to unseen math problems, not just memorize solutions. `parallel=True` with `max_workers=32` enables concurrent evaluation across problems. `cache_evaluation=True` avoids re-evaluating the same prompt on the same problem.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig
from examples.aime_math.dataset import load_math_dataset()

trainset, valset, testset = load_math_dataset()
```

```
result = optimize_anything(
    seed_candidate=SEED_PROMPT,
    evaluator=evaluate,
    dataset=trainset,
    valset=valset,
    config=GEPAConfig(
        engine=EngineConfig(max_metric_calls=600, parallel=True, max_workers=32,
cache_evaluation=True),
    ),
)
```

Optimized artifact — From a single generic sentence, GEPA evolved a structured reasoning framework with explicit heuristics for competition math: restating the problem, defining variables and constraints upfront, justifying each algebraic step, naming theorems before applying them, and backtracking cleanly from dead ends. The prompt also enforces discipline — preferring exact algebra over approximation, testing candidate values only after deriving tight constraints, and isolating the final answer on its own line.

```
Solve the math problem carefully and thoroughly. Your goal is to produce a correct,
well-structured solution that leads unambiguously to the requested final result.

Follow these rules:
```

```
1. Restate the problem briefly in your own words.

2. Set up notation and equations cleanly before manipulating them.
   - Define variables explicitly.
   - State all constraints (e.g., integrality, ranges, geometric conditions) before
     using them.

3. Show clear, logically ordered reasoning.
   - Justify each important algebraic or geometric step.
   - When you split into cases, state why each case is necessary and what
assumptions
     define it.
   - If you invoke a known theorem (e.g., Ptolemy, Power of a Point, similarity,
Vieta),
     name it and show exactly how it applies in this context.

4. Handle dead ends correctly.
   - If you realize a line of reasoning leads to a contradiction or dead end,
explicitly
     say so.
   - Then restart from the last correct point; do not guess or hand-wave.

5. Keep the reasoning focused and minimal while still being rigorous.
   - Avoid unnecessary numerical approximations if an exact approach is available.
   - Do not approximate exact values unless the problem explicitly asks for a
decimal.
   - Prefer algebraic or structural arguments over trial-and-error or random
guessing.
```

```
   - You may test candidate values only after deriving strong constraints that
sharply
     limit the possibilities.

6. At the end, clearly isolate the answer:
   - Provide the final answer as a single number or expression on its own line.
   - Do not include any extra words, symbols, or explanation on that final line.
```

CUDA Kernel Generation (KernelBench)

We tackle KernelBench, a benchmark of PyTorch neural-network operations where the goal is to generate a custom CUDA kernel with C++ extensions that is both correct and faster than the reference PyTorch implementation. A single shared prompt is optimized across multiple problems so that insights from one kernel transfer to others.

Candidate — The seed candidate is a minimal one-line instruction prompt that tells the LLM to generate a CUDA kernel replacement. It provides just enough structure to define the expected output format (a complete Python file using `load_inline`).

```
SEED_PROMPT = """Write a CUDA kernel to replace the given PyTorch model for better performance.
Output a complete Python file with ModelNew using load_inline. Include all imports."""
```

Evaluator — The evaluator compiles the LLM-generated kernel, benchmarks it against a baseline PyTorch implementation for the given problem, and returns rich Actionable Side Information (ASI)

with the generated code, CUDA documentation consulted during generation, runtime measurements, speedup ratios, correctness status, and detailed error feedback when compilation or validation fails.

```
def evaluate(candidate, example):
    baseline = baselines[example.problem_id]
    code, cuda_docs, eval_result = run_kernel(
        candidate, example.ref_arch, lm, predictor
    )
    score = compute_score(eval_result, baseline)

    runtime = eval_result.get("PerformanceStatsMean")
    return score, {
        "score": score,
        "problem_id": example.problem_id,
        "level": example.level,
        "baseline_ms": baseline,
        "code": code,
        "cuda_docs": cuda_docs,
        "cuda_docs_post": post_docs,
        "runtime_ms": runtime,
        "speedup": baseline / runtime if runtime else None,
        "compiled_successfully": eval_result.get("CompilationSucceeded", False),
        "ran_without_error": eval_result.get("NoRuntimeErrorDuringCorrectnessCheck",
False),
        "output_values_correct": eval_result.get("CorrectnessSucceeded", False),
        "error_type": eval_result.get("ErrorType"),
```

```
        "error_detail": eval_result.get("ErrorDetail"),
    }
```

Optimizer — This is a Multi-Task Search: a `dataset` of multiple KernelBench problems means insights from optimizing one kernel transfer to others via shared prompt improvements. `RefinerConfig()` enables automatic per-evaluation refinement — after each evaluation, an LLM proposes a refined candidate based on the feedback. `background` is used to inject CUDA best practices and constraints into the optimization loop.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig,
RefinerConfig

optimize_anything(
    seed_candidate=SEED_PROMPT,
    evaluator=evaluate,
    dataset=dataset,  # KernelBench problems
    config=GEPAConfig(
        engine=EngineConfig(max_metric_calls=2000, cache_evaluation=True),
        refiner=RefinerConfig(),  # auto-refine after each evaluation
    ),
    objective="Generate an LLM prompt that produces fast, correct CUDA kernels
outperforming PyTorch baselines.",
```

```
    background=BACKGROUND,
)
```

Optimized artifact — Below is the best kernel discovered for a LayerNorm CUDA kernel, which led to 3.32x speedup. It loads four numbers at a time from GPU memory instead of one (float4 vectorization), cutting memory transaction overhead by roughly 4x. It splits the work into two clean passes: first compute the mean and variance, then normalize. This lets the GPU optimize each step independently rather than juggling both at once. Finally, threads share their partial sums using direct register-to-register transfers (warp shuffles), skipping the usual detour through slower shared memory.

```
import math
import torch
import torch.nn as nn
from torch.utils.cpp_extension import load_inline

_kb_layernorm_mod = None

def _get_kb_layernorm_mod():
    global _kb_layernorm_mod
    if _kb_layernorm_mod is not None:
        return _kb_layernorm_mod
```

```
    if not torch.cuda.is_available():
        return None
    try:
        _kb_layernorm_mod = load_inline(
            name="kb_layernorm_vec",
            cpp_sources=r"""
#include <torch/extension.h>
```

Circle Packing (n=26)

We tackle circle packing, a classic combinatorial optimization problem benchmarked by many LLM-based evolution algorithms including ShinkaEvolve, OpenEvolve, and AlphaEvolve. The goal is to pack n=26 circles within a unit square, maximizing the sum of radii.

Candidate — We adopt a seed candidate from ShinkaEvolve's code. It places circles in concentric rings: one at the center, a ring of 8 around it, and an outer ring for the remaining circles. It clips positions into the unit square and greedily shrinks radii until no circle overlaps another or breaches a boundary.

```
SEED_CODE = '''
import numpy as np

def main(timeout, current_best_solution):
    """
    Circle packing optimization.

    Args:
        timeout: Time budget in seconds
        current_best_solution: Previous best circles array (n, 3) or None
```

```
    Returns:
        dict with 'circles' (n, 3) array and 'all_scores' list
    """
    n = 26

    # Use current_best_solution if provided, otherwise start fresh
    if current_best_solution is not None:
```

Evaluator — The evaluator sandboxes the proposed code, runs it with a 600-second timeout, validates circles against geometric constraints (no overlaps, all within the unit square), and returns rich ASI with circle positions, multiple metrics derived from evaluation trajectory, validation details, stdout/stderr so the LLM understands which geometric constraints are binding. The `extract_best_circles(opt_state)` helper warm-starts each new proposal with the best circles discovered so far.

```python
from examples.circle_packing.utils import extract_best_circles, execute_code

def evaluate(candidate, opt_state):
    warm_start = extract_best_circles(opt_state)
    result = execute_code(candidate, 600, warm_start)

    circles = result["circles"]
    score = result["validation_details"]["sum_radii"]
```

```
    metrics = compute_multiple_metrics(result["all_scores"])

    side_info = {
        "scores": {"sum_radii": score},
        "metrics": metrics,  # mean, min, max, EMA1, EMA2 of the circles explored by
this candidate
        "code": candidate,
        "circles": circles,
        "stdout": result.get("stdout", ""),
        "error": result.get("error"),
        "traceback": result.get("traceback"),
        "validation_details": result.get("validation_details"),
    }

    return score, side_info
```

Optimizer — This is a Single-Task Search with no `dataset`. `frontier_type="objective"` tracks the Pareto frontier per objective metric (e.g., `sum_radii`) rather than per dataset example. `RefinerConfig()` injects a `refiner_prompt` into a candidate dictionary to add an inner-loop refinement step after each evaluation. A Refiner LLM reads the feedback and proposes an improved candidate. GEPA keeps whichever is better, and the refiner's own prompt is co-evolved alongside the candidate so refinement quality improves over time. Combined with `frontier_type="objective"`,

the `RefinerConfig()` doubles the pareto metrics: the original candidates' metrics + the refiner's metrics.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig,
ReflectionConfig

optimize_anything(
    seed_candidate=SEED_CODE,
    evaluator=evaluate,
    config=GEPAConfig(
        engine=EngineConfig(max_metric_calls=150, cache_evaluation=True,
frontier_type="objective"),
        reflection=ReflectionConfig(reflection_lm="openai/gpt-5"),
        refiner=RefinerConfig()
    ),
    objective="Optimize circle packing code to maximize sum of circle radii "
              "within a unit square for N=26 circles.",
)
```

Optimized artifact — From a 75-line concentric-ring heuristic, GEPA evolved a 900+ line bilevel optimizer that formulates packing as an LP over radii and uses dual variables to derive exact gradients for L-BFGS-B over center positions. It layers SLP with adaptive trust regions, CMA-ES for

evolutionary exploration, and diverse seeding strategies (hexagonal grid, farthest-point insertion, corner spokes, edge rings). The result scores 2.63598+, beating AlphaEvolve's 2.6358.

```python
import numpy as np
import time

def main(timeout, current_best_solution):
    """
    Breakthrough optimizer (refined): Bilevel L-BFGS with exact LP sensitivities
+ SLP block boosts + CMA/Evolution fallback
    - Uses LP over radii with dual-based gradient for centers (bilevel).
    - L-BFGS-B over centers, with proper best-tracking inside objective.
    - Block SLP trust-region boosts on worst circles.
    - Aggressive relocation of worst circles to edges/corners.
    - Multiple diverse seeding + optional CMA-ES and evolutionary exploration.
    - Minor performance/robustness adjustments based on evaluation.
    """
    n = 26
    start_time = time.time()
    time_budget = max(1.0, float(timeout))
    rng = np.random.default_rng(20260124)
```

Blackbox Mathematical Optimization (EvalSet)

Here, we optimize a blackbox solver to minimize polynomial benchmark functions from the Evalset suite, benchmarked by Optuna.

Candidate — The seed candidate is a minimal random-search solver: it samples a single point uniformly at random within the given bounds, evaluates it, and returns the result. This provides a starting point and a template for the API.

```
SEED_CODE = """
import numpy as np

def solve(objective_function, config, best_xs=None):
    bounds = np.array(config['bounds'])
    x = np.random.uniform(bounds[:, 0], bounds[:, 1])
    score = objective_function(x)
    all_attempts = [{"x": x.copy(), "score": score})]

    return {"x": x, "score": score, "all_attempts": all_attempts}
"""
```

Evaluator — The evaluator sandboxes the proposed solver code, runs it on a benchmark function, and returns rich **Actionable Side Information (ASI)**: trial attempts, stdout/stderr, tracebacks, and budget status. A `BudgetTracker` helper distributes whatever evaluation budget is left across the remaining proposals, accounting for failures gracefully. `opt_state` passed through the `evaluate` function tracks the full optimization state; here we extract the best trials so far for warm-starting new proposals.

```
from gepa.examples.mathematical_optimization.utils import BudgetTracker

num_proposals = 10
budget = BudgetTracker(total_budgets=2000)

def evaluate(candidate, opt_state):
    if budget.remaining <= 0:
        return -1e9, {"score": -1e9, "Error": "No budget remaining"}

    code = candidate["code"]
    best_xs = extract_best_xs(opt_state)  # warm-start from prior evaluations
    result = execute_code(code, problem_index=0, budget=100, best_xs=best_xs)
    budget.record(result)

    return result["score"], {
        "score": result["score"],
        "all_trials": result["all_trials"],
```

```
        "stdout": result.get("stdout", ""),
        "error": result.get("error", ""),
        "traceback": result.get("traceback", ""),
        "budget_total": budget.total,
        "budget_used": budget.used,
        "proposal_total": num_proposals,
        "proposal_completed": budget.candidates,
    }
```

Optimizer - This is a Single-Task Search mode: no `dataset` needed, just a natural-language `objective`. `background` lets you inject domain knowledge and constraints into the optimization loop. `cache_evaluation` caches scores and side information, saving metric calls on repeated evaluations. `max_candidate_proposals` caps the total number of proposals generated across the entire run.

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig,
ReflectionConfig

optimize_anything(
    seed_candidate={"code": SEED_CODE},
    evaluator=evaluate,
    config=GEPAConfig(
        engine=EngineConfig(max_candidate_proposals=10, cache_evaluation=True),
```

```
        reflection=ReflectionConfig(reflection_lm="openai/gpt-5"),
    ),
    objective="Evolve Python code that minimizes a blackbox objective function "
              "using the available evaluation budget efficiently.",
    background="# Optional background, code requirements, strategies, etc..."
)
```

Optimized artifact — Here is the best artifact produced for P31: McCourt20. 250+ lines of solver code, fully generated by `optimize_anything`.

```
import numpy as np
from scipy.optimize import minimize
from scipy.spatial.distance import cdist

try:
    from sklearn.gaussian_process import GaussianProcessRegressor
    from sklearn.gaussian_process.kernels import Matern, ConstantKernel,
WhiteKernel
    SKLEARN_AVAILABLE = True
except Exception:
    SKLEARN_AVAILABLE = False


def solve(objective_function, config, best_xs=None):
```

```
    bounds = np.array(config['bounds'], dtype=float)
    dim = config.get('dim', bounds.shape[0])
    budget = int(config.get('budget', 100))
```

Appendix: Make `printf()` Debugging Cool Again

If you already have `print()` statements scattered through your evaluation code for debugging (or perhaps call a library that logs to std streams), you can turn them into ASI with a single flag, no code changes needed:

```
from gepa.optimize_anything import optimize_anything, GEPAConfig, EngineConfig

result = optimize_anything(
    ...,
    evaluator=evaluate,  # existing print() calls become ASI
    config=GEPAConfig(engine=EngineConfig(capture_stdio=True)),
)
```

With `capture_stdio=True`, any `print()` output inside your evaluator is automatically captured and included in `side_info` as `"stdout"` and `"stderr"`, while still printing to your terminal as usual. Your debugging workflow stays the same, but now the optimizer can read the same diagnostics you can. This is

handy for quick prototyping or wrapping existing evaluation scripts. For production use, we recommend `oa.log()` or structured `side_info` dicts for cleaner separation between debugging output and optimization feedback.

---

1. See OpenEvolve's island config (e.g. in their circle-packing example) and ShinkaEvolve's island strategies (e.g. in their circle-packing example). ↩
2. See OpenEvolve's prompt sampler and template system and ShinkaEvolve's patch-type sampling with prompt co-evolution. ↩
3. See OpenEvolve's three-stage cascade evaluator (e.g. configured here) and ShinkaEvolve's multi-run evaluation wrapper. ↩